

Capstone

SOEN 490

Peter C Rigby

PhD, P.Eng (Ontario)

Undergraduate Program Director, Software Engineering

Professor, Concordia University

<https://users.encs.concordia.ca/~pcr/>

[Book](#) time to meet with me

To pass 490

You must demonstrate **“engineering design and implementation for complex, open-ended engineering problems.”**

This is required by the CEAB (Canadian Engineering Accreditation Board)

Have Fun!

Most software engineering work involves maintenance of an existing system.

SOEN 490 is your chance to do something innovative without having a large company and boss restricting you.

11.25 hours per week on SOEN 490

- "Each credit represents a minimum of 45 hours of academic activity, including lectures, tutorials, laboratories, studio or practice periods, examinations, and personal work."
- So for SOEN490, it represents $45 \times 6 = 270$ hours per student.
- 490 is more flexible, but don't let it slide for more than a week.
- Part of the grade is consistent work over the entire two terms.

Goal

- Make capstone as realistic as possible
- Start Agile, deal with risks that would kill a project
- Use continuous integration
- Replace agile with live system and logs

Fail fast

- What risk would kill your project?
- AKA get rid of risks without writing any production code
- Pipeline for [data projects](#)
- Risks should be domain specific
 - Ex. “we may not get a critical mass of users”
- Do not write down general SE risks that all project have:
 - Ex. don’t write “the team might not meet deadlines.”

Students struggling with 490

- Everyone in 490 must demonstrate their own "substantial engineering design and implementation".
- Some students do not know the libraries, language, or other important parts of the system as well as others on the team. In this case:

1. You should talk with your team lead and take on low risk, low priority stories

2. This story should not be on the critical path (if it is someone else will make sure it gets done)

3. Make it clear at the top of the story that other people on your team shouldn't finish this story for you.

- **Please do NOT add yourself to stories that you didn't do real work on. Find stories you can own and demonstrate engineering work.**

AI code assistants

- These are vital and helpful tools
- You must attribute their contribution in the commit message, for example:
 - “CoPilot helped identify the correct API and usage”
- Failure to attribute AI tooling will be considered plagiarism
 - “My teammate keeps submitting AI crap and doesn’t even acknowledge it”
 - I have to investigate this, the punishment goes outside of my hands, [Academic Offenses](#)

Proposal

- Fill in this [document](#)
- Only send me one completed proposal at a time
- Potential projects are first come first served
- Your GitHub Repo name is your ProjectName and is written in CamelCase. Your team name is your ProjectName
- Once I approve your project, I will send you a link to upload your GitHub url and username
 - Grading revolves around your GitHub username

No Stakeholder, startup

- You can run your project as a startup without an external stakeholder
- This is not the preferred mode of 490
- You will need to be doing something novel/disruptive
- You must describe how you tried to find a stakeholder and why you can't find one
- You must have a plan to get early user feedback by Release 1 (in the Fall)
- You must describe how you will get additional user feedback/alpha release by Release 2 (end of January)

Legal Aspects

- **I'm not a lawyer and this is not legal advice**
- **I will not give you a grade for Release 1 until you have done the following:**
- Liability disclaimer
 - **I don't want you to get sued**
- IP and Copyright
- End User License Agreement
- The source code license is on GitHub
- Stakeholder agreement

Liability

- All projects must include the following (even if you have a stakeholder)
- THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Copyright and IP

- If stakeholder owns IP and you transfer copyright
 - Do you really want to give all your work to the stakeholder?
- A lack of formal IP agreement is terminal for a project
 - This is unacceptable for 490
 - You can get a lawyer to write a license (will cost too much)
- Recommended
 - Use the [MIT license](#)

MIT License

- You overvalue your ideas. Anything that a capstone team can write in 8 months while taking other courses an experienced professional can rewrite in a month
 - Basically, you have something that could get you venture funding to continue working on the code
- MIT
 - Anyone who has the source code can do anything they want with it including selling it and incorporating it into commercial private code
 - You just can't sue the authors

IP opt-out

- Sign Concordia's [IP opt-out](#)
 - Concordia doesn't own student work
 - Concordia is not liable for student work
 - Can't sign this if your Stakeholder is an employee of Concordia, eg a prof
 - Use and MIT license

NDA

- Ugh, ugh, ugh
- Only if your stakeholder requires it
- You can sign any NDA you want, but be careful, the company will usually own all your ideas (even outside of capstone)
- I can't sign this type of NDA, my NDA is for grading purposes (the dean actually signs it)
- **There is only one NDA I can sign.**
- If your stakeholder doesn't agree to it by the end of September, **your project is cancelled** and I will assign you a new project

End user license agreements

- EULA
- These are really hard to write
- I'm not a lawyer and this is not legal advice
- Find a project/company that is similar and adapt theirs
 - Eg [Airbnb](#)
- You must have users agree at the beginning
- You can get ChatGPT to help
 - Not the same as a lawyer, but probably better than nothing

Stakeholder Responsibilities

- Get your stakeholder to fill in the [Stakeholder responsibilities](#) which also requires a liability limitation clause in the software license.
- This must be done before I approve the project

GitHub

- Code is in git
- Features: user stories and log feedback will be tracked as GitHub issues
 - Only fix bugs that relate to high priority features
- This [doc](#) explains how
- Add rigbypc as an editor to your project
- By law, I will need access for 1 year after the project finishes in the event of a grade dispute

You must

1. You must have CI from the beginning (GitHub actions)
2. You must have logging from the beginning
3. You must have a unit tests and a mocking library
4. You must containerize everything

Containerization

- Encapsulate your applications (and supporting components) into containers
- Distribute and ship those containers to your teams for further development and testing
- Deploy those applications to your production environment, whether it is in a local machine or the Cloud
- We will use Docker. If you want to use another container solution please write me a justification

You should manage it

1. Everything must be containerized
2. You must host/manage the service in a container
 - a. eg. No firebase dbs

Or

3. If you can't host the service, write a one paragraph justification
 - a. Will also need to describe the SE effort
 - b. Image processing, translation, AI API
 - c. I'm flexible, don't let this stop you from doing awesomeness, just tell me why you need it

You should have

1. You should submit all your changes through pull requests (ie no direct commits)
 - a. At least one person should review
2. You should avoid branches
3. You should have a toggle library
4. ***Important* make sure your GitHub username is linked correctly so I can see the commits you made by clicking on your username**

Personas

- A persona is a faked person that describes a class of users you will design for
- Can replace “user” with the persona’s name in user stories
- You can also describe personas that you explicitly won’t design for (helps scope the project)
- Some helpful [advice](#)

Agile Planning and Estimating

Peter C Rigby

Kent Beck Lecture

- You register [here](#) and then you can watch the video
- “make design decisions later and in small, safe steps.”

Text Book

- Highly recommended that you get and read this book:
- “Agile Planning and Estimating,” by Mike Cohn
- [Available](#) online from the library

Key points

- Use **relative** size estimates
- **Derive** time estimates based on velocity
- Spend a small amount of time on estimating
- Use different planning horizons
 - Day, Iteration (2 weeks), Release (2 times), Final Product Release (In April)

Stories (Ch4)

- A story of how the system will be used
 - As a user, I can do blah, so I can ...
- A story is not a task, an algorithm, etc
- A feature or epic is broken down into multiple stories

490 Story example

- As a user, I want to manipulate the inventory, so I can select an item
- [link](#)

Story Points

- The size or difficulty (not the time) is estimated relative to other stories on the following scale
- Story: 0, 1, 2, 3, 5, 8
- Epic: 13, 20, 40, 100

<https://daily.dev/blog/t-shirt-sizing-in-agile-guide-2024>

Size	Typical Meaning
XS	Very quick task
S	A few hours of work
M	About a day's work
L	Up to a week's work
XL	More than a week's work

Planning Poker (Ch6)

- Moderator reads user story
- Each developer chooses a card and places it facedown
- Turn cards
- Highest and lowest estimate explain rationale
- Return to step two until consensus
- (Can use phone Apps instead)

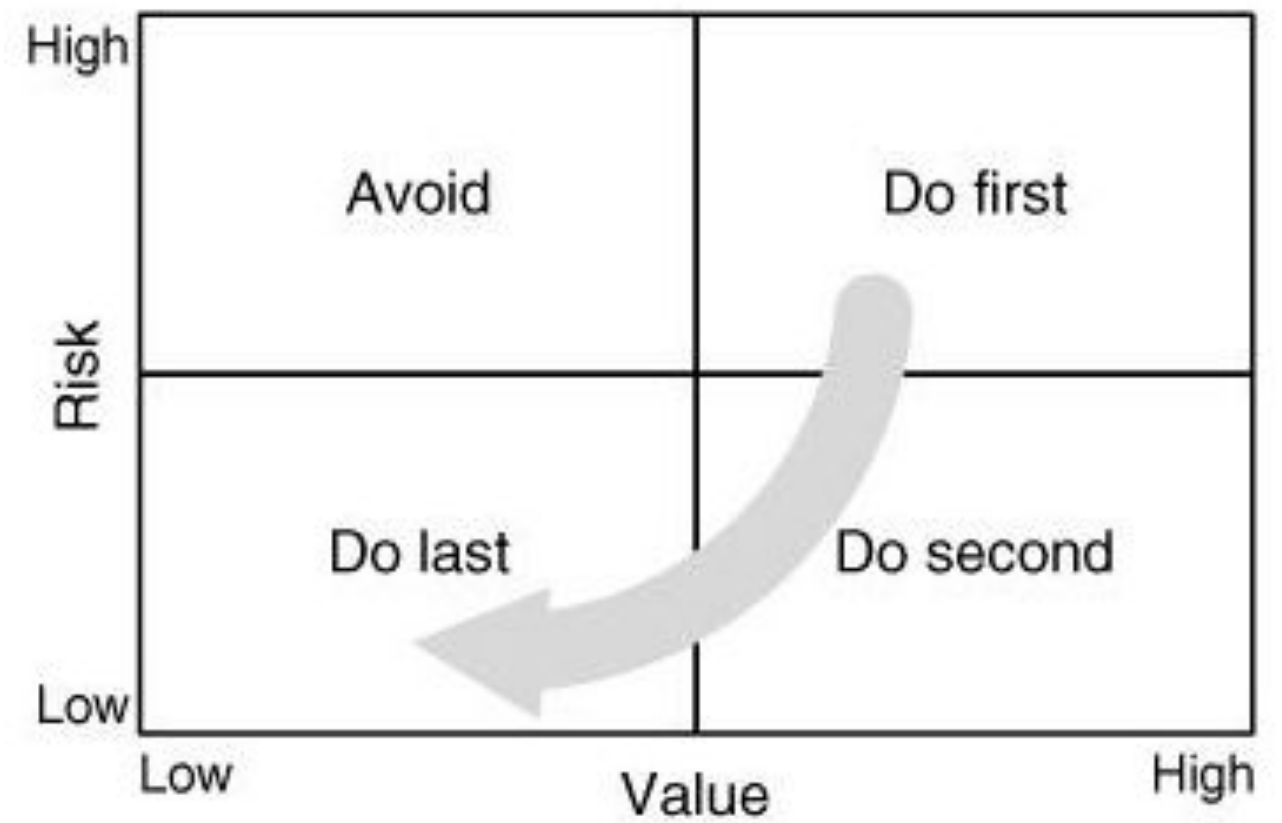
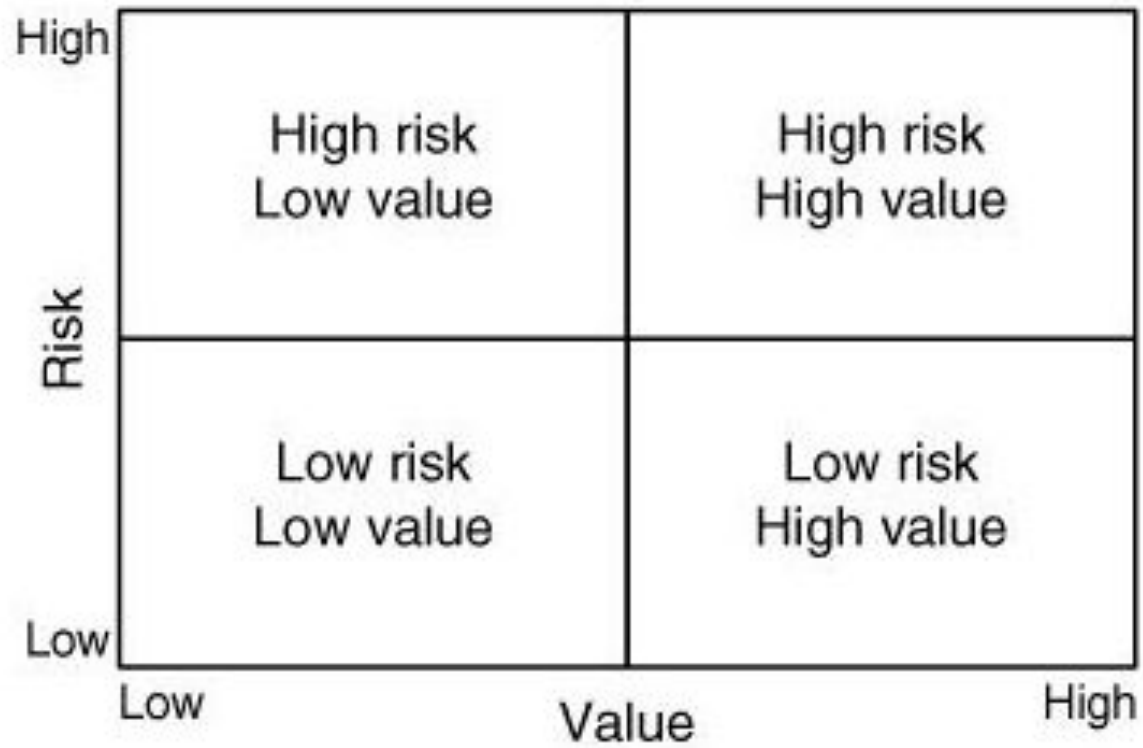
490 Feature labels example

- [Features](#)
 - [Combat system](#)
 - [HUD - Heads-up Display](#)

Prioritizing (Ch 9)

- Stories are prioritized by the discussions with the customer
- but don't forget to add risk

Risk



Horizons

- Day
- Iteration (Set of stories)
- Release (2 releases)
- Product (April)
- Milestones in github, see [490 example](#)

Iteration Plan

- Select the stories you're going to finish in the iteration
- Break the story down into task, with ideal hour estimates
- Exit Criteria:
 - Story has demo steps, stakeholder signoff, and is tested

Task

- Estimated in ideal hours
- [490 example](#)

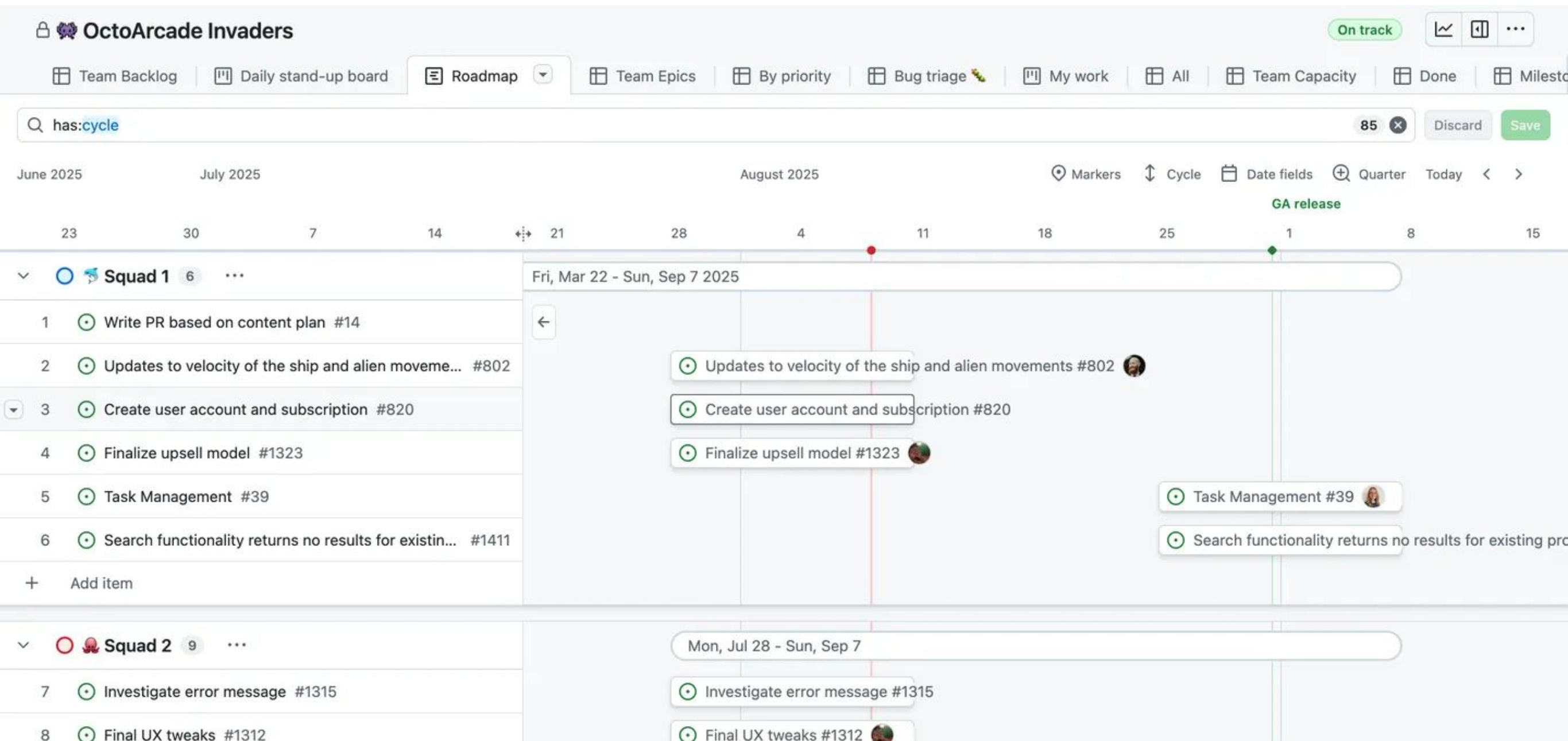
Key points

- Use **relative** size estimates
- **Derive** time estimates based on velocity
- Spend a small amount of time on estimating
- Use different planning horizons
 - Day, Iteration (2 weeks), Release (2 times), Product (In April)

Iterations and releases

1. You will work in two week iterations
2. You will plan two iterations out
3. There will be three releases
 - a. Release 1: 30%, due Nov 10th
 - b. Release 2: 30%, due Jan 26th
 - c. Final Release and presentation: 40%, due April 13th. Note: You must pass the final release to pass the course.
4. [Schedule](#)
5. [Course outline](#)

GitHub project boards



Your GitHub Wiki

Readme

Release demos

Project summary (max one paragraph)

Developer getting started guide

Wiki table of contents

Wiki

Meeting Minutes

Risks

User consent and end-user license agreement

Legal and Ethical issues

Economic

Budget

Personas

Diversity statement

Overall Architecture and Class Diagrams

Infrastructure and tools

Name Conventions

Testing Plan and Continuous Integration

Security

Performance

Deployment Plan and Infrastructure

Missing knowledge and Independent Learning

- Everything will be on GitHub, including your documentation, design, and readme

Keep your receipts

- The university will reimburse around \$200
- I will send the procedure next term

Problems with Agile

- The stakeholder is not the actual user or is only one of the users
- The real priority is unclear
- The real risk is unclear
- The real estimate for task length is unclear

What is important?

- Are priority, risk, and size important?
- We need data from users!

Pick a Real metric

- Money, clickthroughs, speed of transactions

Continuous delivery and Logs

- Release continuously to a small user base (alpha)
- Collect logs and extract user metrics
- User metrics drive features and bug fixes
- [A/B testing](#) allows you to decide among features

Empirical Data

- Features are prioritized based on your key metric
- Probably: number of users
- Don't work on features that don't have users
- Fail fast: release code quickly and iterate

But who will use your
software?

Guerrilla Marketing

- Low budget and unconventional marketing
- Athletics software
 - Stand outside Le Gym and try and get people to signup
- Food delivery software
 - Get a booth at a foodie show or go to a foodie meetup
- After release 2, you will do this!

User studies

- Keep in mind your end user license agreement
- If you aren't comfortable, talk with me about how to do user studies instead.

Bug Bash Iteration

- Find some “users” to heavily use your system
- Report and prioritize the bugs
- Fix the most important ones
- Increasingly seen as a good substitute for UI automated testing

Iteration and release notes

- See this [document](#)
- You can begin with automated ones
- Each [individual](#) must describe what they have done
 - This is visible to the team lead and your teammates, and it is part of the team leads job to point out inappropriate claims to work
- You will also discuss went well and what didn't
- 490 is iterative, if you don't do any work for an iteration you will get zero for that iteration

Release Presentations

[Slide deck](#) to copy and instructions

Grading Scheme

- [Tentative Grading Scheme](#)
- I will grade all the projects
- TAs will do a deep dive into the code
- Peer and other individual evaluations may mean different grades for teammates
- You must be able to show your own “substantial engineering design and implementation”.

Peer Evaluations

- One per release
- When looking for problematic teammates who have not done enough work, **I only look at the negative scores**
- If you have consistent negative scores, you will have an opportunity to send me a video walking me through your substantial engineering design and implementation
- I will check this against commits, issues, and the release/iteration note, and peer review comments (including positive ones)

Capstone Awards

- People's Choice Award
 - Team with the most votes for best project at the EV building Capstone poster/dem showcase
- Deployment Award. An important part of release 3 is to show the level of deployment and feedback that you received. I will potentially give out multiple awards, eg most users vs deployed internally at a company.

- Be original, use modern technology, and techniques from workterms
- There are some limited machines and resources available through the university. Please contact me if you want a physical machine.
- You will need to show real SE effort that goes beyond plugging libraries together and code that an AI can generate, ie it needs to be substantial

Anonymous Feedback

- Form [here](#)
- You can provide me any feedback you want. Email addresses aren't kept. You can fill it in as many times as you like. Please avoid information that will reveal yourself. Please also try and be constructive (eg suggest a solution).
- I'm happy to change things that have a good reason
 - Eg I changed to bookable office hours because people got tired of waiting

Software “Laws”

Slides here